

DOCUMENTAÇÃO TÉCNICA

Processador MIPS

Sumário

- 1. Visão Geral da Arquitetura
- 2. Diagrama de Blocos
- 3. Módulos do Processador
 - 3.1 PC — Program Counter
 - 3.2 Memória de Instrução (ROM)
 - 3.3 Splitter de Instrução
 - 3.4 Unidade de Controle
 - 3.5 Banco de Registradores
 - 3.6 ULA — Unidade Lógica e Aritmética
 - 3.7 Controle ULA
 - 3.8 Memória de Dados (RAM)
 - 3.9 Multiplexadores
 - 3.10 Lógica de Desvio e Salto
- 4. Conjunto de Instruções (ISA)
- 5. Tabela de Sinais de Controle
- 6. Decisões de Projeto e Justificativas
- 7. Limitações e Possíveis Melhorias

1. Visão Geral da Arquitetura

Este documento descreve um processador MIPS simplificado de 8 bits, implementado em Logisim com arquitetura monociclo. O processador foi projetado para fins didáticos, mantendo os princípios fundamentais da arquitetura MIPS enquanto adapta parâmetros de largura de dados para facilitar a visualização e depuração no simulador.

Especificações Gerais

Arquitetura: Monociclo (MIPS simplificado)
Largura de dados: 8 bits
Endereçamento: 8 bits (256 posições de memória)
Formato de instrução: 18 bits
Registradores: 8 registradores de propósito geral (R0–R7)
Memória de instrução: ROM 8x18 bits
Memória de dados: RAM 8x8 bits separada
Ferramenta: Logisim Web

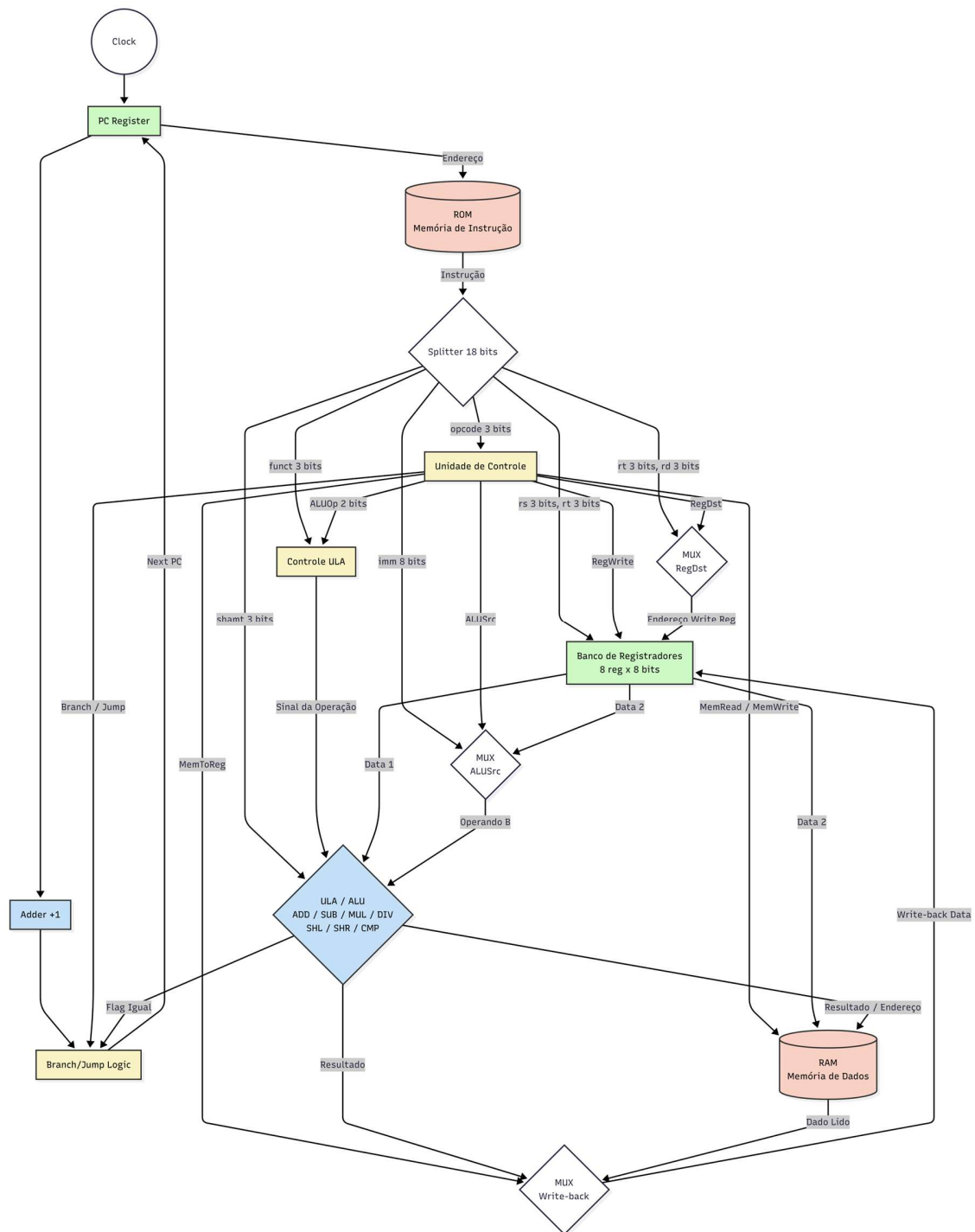
A arquitetura segue o modelo clássico de 5 estágios do MIPS, porém executados em um único ciclo de clock:

- IF — Instruction Fetch (busca de instrução na ROM)
- ID — Instruction Decode + Register Read (decodificação e leitura de registradores)
- EX — Execute (execução na ULA)
- MEM — Memory Access (acesso à RAM de dados)
- WB — Write Back (escrita de resultado no banco de registradores)

Por ser monociclo, todos os estágios ocorrem dentro do mesmo período de clock, sem pipeline. Isso simplifica o controle mas limita o desempenho em frequências mais altas.

2. Diagrama de Blocos da Arquitetura

O diagrama abaixo representa o fluxo de dados e sinais de controle entre os módulos principais do processador:



Legenda dos Sinais de Controle

RegDist — Selecciona entre rt (I-type) e rd (R-type) como registrador de destino
ALUSrc — Selecciona entre leitura do Reg2 e imediato como segundo operando da ULA
MemWrite — Habilita escrita na RAM
MemRead — Habilita leitura na RAM
MemToReg — Selecciona resultado da ULA ou dado da RAM para write-back
RegWrite — Habilita escrita no banco de registradores
Branch — Sinaliza instrução de desvio condicional (BEQ)
Jump — Sinaliza salto incondicional
ALUOp — Código 2 bits que define a operação da ULA

3. Descrição dos Módulos

3.1 PC — Program Counter

O PC é implementado como um registrador de 8 bits (componente Register do Logisim, rotulado "PC"). Sua função é armazenar o endereço da instrução em execução e, a cada borda de subida do clock, atualizar-se com o próximo endereço.

Lógica de atualização do PC:

- Caso normal: $PC \leftarrow PC + 1$ (somador dedicado com constante 1)
- Branch tomado: $PC \leftarrow PC + 1 + \text{offset}$ (quando Branch AND Zero)
- Jump: $PC \leftarrow \text{endereço imediato da instrução}$

O mux final seleciona entre as três fontes por meio dos sinais Branch, Jump e da flag Zero da ULA, processados via porta AND (Branch & Zero) e porta OR com Jump.

3.2 Memória de Instrução (ROM)

A memória de instrução é uma ROM de 8 endereços $\times$ 18 bits de dados. O endereço é fornecido diretamente pelo PC. Cada posição armazena uma instrução completa de 18 bits codificada em hexadecimal.

Parâmetro	Valor
Tipo	ROM (somente leitura)
Largura de endereço	8 bits (256 posições máximo)
Largura de dados	18 bits por instrução
Conteúdo inicial	0x38001 (instrução de teste)
Latência	Combinacional (monociclo)

A ROM é de acesso assíncrono: a instrução fica disponível na saída imediatamente após o PC apresentar o endereço, sem necessidade de clock separado.

3.3 Splitter de Instrução

Após a saída da ROM, um Splitter de 18 bits decompõe a instrução nos seus campos:

Campo	Bits	Largura	Uso
opcode	[17:15]	3 bits	Tipo de instrução $\rightarrow$ Unidade de Controle
rs	[14:12]	3 bits	Registrador fonte 1 $\rightarrow$ Banco de Registradores
rt	[11:9]	3 bits	Registrador fonte 2 / destino I-type
rd	[8:6]	3 bits	Registrador destino R-type

Campo	Bits	Largura	Uso
shamt	[5:3]	3 bits	Deslocamento (shift amount) → ULA
funct	[2:0]	3 bits	Código de função R-type → Controle ULA
imm / addr	[7:0]	8 bits	Imediato / endereço (I-type e J-type)

Os campos de 3 bits (opcode, rs, rt, rd, shamt, funct) são extraídos por splitters separados, cada um conectado via Tunnels aos módulos correspondentes, evitando fios longos e cruzamentos visuais.

3.4 Unidade de Controle

A Unidade de Controle é um módulo combinacional que recebe o opcode de 3 bits e gera 9 sinais de controle que governam o comportamento de todos os outros módulos no ciclo corrente. Internamente, é implementada com portas lógicas AND, OR, NOR e XOR.

Estrutura interna:

O opcode de 3 bits é expandido por um Splitter em 3 linhas individuais (bits 0, 1, 2). Cada sinal de controle é derivado por uma expressão booleana específica implementada com as portas disponíveis. O sinal ALUOp é de 2 bits e controlado por um Splitter de saída.

Sinal	Tipo	Descrição
RegDist	1 bit	0=rt é destino (I-type), 1=rd é destino (R-type)
ALUSrc	1 bit	0=usa Reg2, 1=usa imediato como operando B da ULA
MemWrite	1 bit	Habilita escrita na RAM (store)
MemRead	1 bit	Habilita leitura na RAM (load)
MemToReg	1 bit	0=resultado ULA vai para Reg, 1=dado da RAM vai para Reg
RegWrite	1 bit	Habilita escrita no banco de registradores
Branch	1 bit	Instrução BEQ — desvio condicional
Jump	1 bit	Instrução J — salto incondicional
ALUOp	2 bits	00=soma, 01=subtração (branch), 10=R-type (usa funct)

3.5 Banco de Registradores

O Banco de Registradores contém 8 registradores de 8 bits (Reg0 a Reg7), implementados com registradores do Logisim. A leitura é assíncrona (dois multiplexadores 8:1 de 8 bits), enquanto a escrita é síncrona (controlada pelo clock e pelo sinal RegWrite).

Portas de interface:

- Read Register 1 (3 bits) → endereço do registrador para Read Data 1
- Read Register 2 (3 bits) → endereço do registrador para Read Data 2
- Write Register (3 bits) → endereço do registrador a ser escrito

- Write Data (8 bits) → dado a ser escrito
- Reg Write (1 bit) → habilita escrita
- CLOCK → borda de subida aciona escrita
- CLEAR → zera todos os registradores
- Read Data 1 (8 bits) → valor do registrador rs
- Read Data 2 (8 bits) → valor do registrador rt

A escrita usa um Demultiplexador 1:8 para rotear o sinal RegWrite ao enable do registrador correto. O seletor Write Register determina qual Reg recebe WE (Write Enable).

3.6 ULA — Unidade Lógica e Aritmética

A ULA é o núcleo computacional do processador. Recebe dois operandos de 8 bits (A e B) e o sinal de controle "Função" de 3 bits oriundo do Controle ULA. Produz um resultado de 8 bits e um flag "Igual" (1 bit) para instruções de comparação e branch.

Operações suportadas:

Código (Função)	Operação	Componente Logisim	Uso típico
000	Soma (ADD)	Adder	add, addi, lw, sw
001	Subtração (SUB)	Subtractor	sub, beq
010	Multiplicação (MUL)	Multiplier	mul
011	Divisão (DIV)	Divider	div
100	Shift esquerda (SHL)	Shifter (left)	sll
101	Shift direita (SHR)	Shifter (right)	srl
110	Comparação (CMP)	Comparator + Bit Extender	slt
111	Negação (NEG)	Negator	neg

Internamente, a ULA utiliza dois Demultiplexadores 1:8 para rotear os operandos A e B para o componente aritmético correto, e um Multiplexador 8:1 na saída para selecionar o resultado da operação ativa. O flag "Igual" é produzido pelo Comparador, estendido de 1 bit para 8 bits via Bit Extender, e também disponibilizado diretamente para a lógica de branch.

3.7 Controle ULA

O Controle ULA recebe dois sinais — ALUOp (2 bits) e Funct (3 bits) — e produz o sinal Função (3 bits) que configura qual operação a ULA irá executar. Esta hierarquia de controle é clássica na arquitetura MIPS.

Lógica de decodificação:

- ALUOp = 00: operação de soma forçada (independe do funct) — usado em load/store e addi
- ALUOp = 01: operação de subtração forçada — usado em branch (beq)

- ALUOp = 10: usa o campo funct para determinar a operação — instruções R-type

A implementação usa um Multiplexador 4:1 de 3 bits selecionado pelo ALUOp. As entradas são: constante 0x0 (soma) para ALUOp=00/11, constante 0x5 (subtração) para ALUOp=01, e o campo Funct diretamente para ALUOp=10.

3.8 Memória de Dados (RAM)

A memória de dados é uma RAM síncrona de 8 bits de endereço e 8 bits de dado, com barramentos de endereço, dado e controle separados (bus=separate). Suporta leitura (MemRead=1) e escrita (MemWrite=1) em ciclos independentes.

Parâmetro	Valor
Tipo	RAM (leitura/escrita)
Largura de endereço	8 bits
Largura de dado	8 bits
Modo de barramento	Separado (address / data-in / data-out)
Clock	Escrita na borda de subida com MemWrite=1
Leitura	Assíncrona quando MemRead=1

O endereço da RAM é o resultado da ULA (calculado como base + offset em instruções lw/sw). O dado de entrada vem do Read Data 2 do Banco de Registradores (registrador rt em sw). O NOT Gate na saída Address inverte o clock para garantir que a escrita ocorra no momento correto em relação ao pipeline monociclo.

3.9 Multiplexadores

O processador utiliza quatro multiplexadores principais para implementar os pontos de decisão do fluxo de dados:

MUX	Largura	Seletor	Entrada 0	Entrada 1	Função
MUX-RegDist	3 bits	RegDist	rt (I-type)	rd (R-type)	Seleciona registrador de destino
MUX-ALUSrc	8 bits	ALUSrc	Read Data 2	Imediato (8 bits)	Seleciona operando B da ULA
MUX-MemToReg	8 bits	MemToReg	Resultado ULA	Read Data (RAM)	Seleciona dado para write-back
MUX-PC	8 bits	OR(Branch&Zero, Jump)	PC+1	Endereço de desvio/salto	Seleciona próximo PC

3.10 Lógica de Desvio e Salto

A lógica de controle de fluxo combina sinais da Unidade de Controle e da ULA:

- Branch condicional (beq): o sinal Branch (da UC) é combinado com o flag Zero/Igual da ULA por uma porta AND. Se ambos forem 1, o desvio é tomado.
- Salto incondicional (jump): o sinal Jump vai diretamente para a lógica do PC, sobrepondo qualquer outro resultado.
- Uma porta OR combina o resultado do AND de branch com o sinal Jump para selecionar o MUX do próximo PC.

O endereço de destino do branch é calculado somando $PC+1$ com o campo de offset (imediato estendido). O endereço de jump vem diretamente do campo imediato/addr da instrução.

4. Conjunto de Instruções (ISA)

O processador suporta os formatos clássicos de instrução MIPS adaptados para 18 bits:

Formato	Campos	Bits
R-type	opcode(3) rs(3) rt(3) rd(3) shamt(3) funct(3)	18 bits

5. Tabela de Sinais de Controle

A tabela abaixo resume todos os sinais gerados pela Unidade de Controle para cada tipo de instrução (determinado pelo opcode de 3 bits):

Instrução	RegDst	ALUSrc	MemWrite	MemRead	MemToReg	RegWrite	Branch	Jump	ALUOp
R-type (000)	1 (rd)	0	0	0	0	1	0	0	10
addi (001)	0 (rt)	1	0	0	0	1	0	0	00
lw / load (010)	0 (rt)	1	0	1	1	1	0	0	00
sw / store (011)	—	1	1	0	0	0	0	0	00
beq (100)	—	0	0	0	0	0	1	0	01
jump (101)	—	—	0	0	0	0	0	1	00
R-type (110)	1 (rd)	0	0	0	0	1	0	0	10

Nota: O campo "—" indica sinal não relevante (don't care) para aquela instrução. R-type cobre opcode 000 e 110 para suportar mais operações aritméticas com o mesmo banco de registradores.

6. Decisões de Projeto e Justificativas Técnicas

6.1 Arquitetura Monociclo

A opção por arquitetura monociclo (em vez de multiciclo ou pipeline) foi a primeira e mais determinante decisão do projeto.

- Justificativa principal: simplicidade de implementação e depuração em Logisim. Em uma arquitetura monociclo, cada instrução produz resultados visíveis imediatamente após o clock, sem a complexidade de registradores de pipeline, hazards de dados ou stalls.
- Contrapartida aceita: o período de clock deve acomodar o caminho crítico (geralmente lw: ROM → Splitter → UC → BancoReg → ULA → RAM → MUX → BancoReg), tornando instruções simples como add mais lentas do que seriam em pipeline.
- Adequação didática: a proposta do projeto é demonstrar o funcionamento completo do processador de forma transparente, o que o monociclo cumpre perfeitamente.

6.2 Largura de 8 Bits

A escolha de 8 bits para dados e endereços (em vez dos 32 bits canônicos do MIPS) foi motivada por:

- Viabilidade visual: fios de 8 bits são muito mais fáceis de acompanhar no Logisim do que fios de 32 bits.
- Recursos limitados do simulador: componentes de 8 bits requerem menos recursos computacionais na simulação.
- Preservação dos conceitos: todos os princípios arquiteturais (banco de registradores, ULA, memórias, controle) funcionam identicamente; apenas a precisão numérica é reduzida.

6.3 Instrução de 18 Bits

O formato de instrução de 18 bits é um compromisso entre:

- Manter os 6 campos do formato R-type MIPS (opcode, rs, rt, rd, shamt, funct), cada um com 3 bits, totalizando 18 bits.
- Compatibilidade com 8 bits de endereçamento nas instruções I-type e J-type, preservando a expressividade para offsets e endereços razoáveis.
- Alternativa considerada: 16 bits (com campos menores) foi descartada por reduzir excessivamente o número de registradores endereçáveis.

6.4 Uso de Tunnels

Extensivo uso de componentes Tunnel do Logisim para conectar sinais entre módulos distantes sem fios longos:

- Melhora significativa da legibilidade do diagrama — fios cruzados são eliminados.
- Facilita manutenção: alterar o roteamento de um sinal requer apenas renomear dois Tunnels.
- Sinais críticos com Tunnels nomeados: RegDist, ALUSrc, MemWrite, MemRead, MemToReg, RegWrite, Branch, Jump, ALUOp, ULACtrl, rs, rt, rd, shamt, funct, opcode, adress/lmm, contRT.

6.5 Banco de Registradores com 8 Registradores

A escolha de 8 registradores (em vez dos 32 do MIPS convencional) foi feita porque:

- Com opcode de 3 bits, apenas 3 bits sobram para endereçar registradores nos campos rs, rt, rd — suficiente para 8 registradores.
- 8 registradores são suficientes para programas didáticos simples.
- A implementação usa demultiplexadores 1:8 e multiplexadores 8:1, componentes diretamente suportados pelo Logisim.

6.6 ULA Modular com Múltiplas Operações

A ULA foi projetada com 8 operações distintas (ADD, SUB, MUL, DIV, SHL, SHR, CMP, NEG) para demonstrar a extensibilidade do design:

- Cada operação usa o componente aritmético nativo do Logisim (Adder, Subtractor, Multiplier, Divider, Shifter, Comparator, Negator).
- Dois demultiplexadores roteiam os operandos A e B para o componente correto; um multiplexador seleciona a saída correta.
- O código de função de 3 bits permite até 8 operações distintas — número adequado para a proposta didática.

6.7 Hierarquia de Controle da ULA

A separação entre Unidade de Controle (que produz ALUOp) e Controle ULA (que produz o sinal Função para a ULA) reproduz fielmente a hierarquia do MIPS convencional:

- Permite que a Unidade de Controle seja mais simples (produz apenas 2 bits de ALUOp).
- O Controle ULA resolve os detalhes de qual operação realizar baseando-se no campo funct (para R-type) ou em operações fixas (para I-type).
- Esta separação é uma decisão de design intencional para demonstrar a estrutura hierárquica de controle presente em processadores reais.

7. Limitações e Possíveis Melhorias

7.1 Limitações Atuais

- Ausência de extensão de sinal: o imediato de 8 bits não é estendido com sinal (sign extension), limitando operações com valores negativos.
- Sem suporte a exceções: não há mecanismo de tratamento de overflow, divisão por zero ou instruções inválidas.
- Memória de instrução pequena: a ROM com 8 posições limita programas a no máximo 8 instruções.
- Sem instrução NOP: não há instrução de no-operation explícita.
- Clock único: todos os módulos compartilham o mesmo clock sem possibilidade de gating.

7.2 Melhorias Propostas

- Pipeline de 5 estágios: adição de registradores de pipeline entre os estágios IF/ID, ID/EX, EX/MEM e MEM/WB para aumentar o throughput.
- Extensão para 16 ou 32 bits: aumentar a largura de dados para maior precisão e compatibilidade.
- Forwarding Unit: para resolver hazards de dados em uma eventual versão pipeline.
- Hazard Detection Unit: para inserir bolhas (stalls) quando o forwarding não é suficiente.
- Extensão de sinal: adicionar um extensor de sinal (sign extender) para o campo imediato.
- Mais registradores: expandir para 16 ou 32 registradores com campos de endereço maiores.
- Memória de instrução maior: substituir ROM por RAM programável para programas mais complexos.